HANDOVER

Running and looking after Batches

What the system runs on, what it costs, how to set it up, keep it healthy and grow it.

For Ladles of Love's IT partners, developers and system owners

SYSTEM	BUILT BY	VERSION
Batches warehouse system	UCT INF3003W Team 22	October 2026

Contents

01	Batches in one page	What it is, the stack at a glance, your first week
02	What it runs on	The parts, the outside services, and what each one needs
03	What it costs	Today's free set-up, its limits, and the recommended live set-up
04	Accounts and access	Every account to hand over, and who should own it
05	Setting it up and going live	From nothing to a working warehouse, step by step
06	Configuration reference	Every setting, where it goes, and what happens without it
07	The codebase	Where things are, the rules the code follows, tests
08	Running it and releasing changes	On your machine, on Render, and the release routine
09	The database	What it holds, changing it, backups, capacity
10	Looking after it	Routine tasks, scheduled jobs, secrets, fixing problems
11	Bringing in another warehouse	Gauteng, Northern Cape and beyond
12	Security and data protection	How the system protects people and data, and your part
13	Growing it	Limits, open decisions, support model, further reading

WHO THIS IS FOR

The developers, IT partners and Ladles of Love staff who host, maintain and extend Batches after Team 22. Sections 1 to 4 need no programming knowledge and are written for whoever at Ladles of Love owns the system. Sections 5 onward are for the person doing the technical work. How the warehouse team uses the system is in the companion document, *How to use Batches*.

01

SYSTEM OWNER

Batches in one page

Batches is the Ladles of Love warehouse system. It runs purchase orders and receiving, stock, decanting, picking slips and packing, dispatch, donations and Section 18A certificates, benevolent requests, Feed the Soil, volunteer events and reporting.

Warehouse staff use it on phones; managers and admins use it in a browser. It is one web application in front of one database, deliberately kept simple so a small team, or a single contractor, can look after it.

Part	What it is
Screens	React 19 and Vite, Tailwind, installable as an app (PWA). Works offline on the floor. English, Afrikaans and isiXhosa on the floor
Server	Node.js 22 and Express 5, one process
Database	PostgreSQL on Supabase (about 88 tables)
Hosting	Render, one web service that serves both the screens and the API
Code	One Git repository on GitHub: `client/`, `server/`, `docs/`
Tests	About 2,100 client and 4,300 server tests, run with `npm test`; GitHub Actions runs checks on every push
Live address today	`https://wms-lol.onrender.com` (free plan, deployed from the `staging/(DEVELOPMENT-TESTING)` branch)

4
roles: warehouse staff, manager, admin, guest volunteer

2
things to host: one web service, one database

≈ $32
per month for a reliable live set-up (section 3)

6
optional outside services, each switches off cleanly

YOUR FIRST WEEK

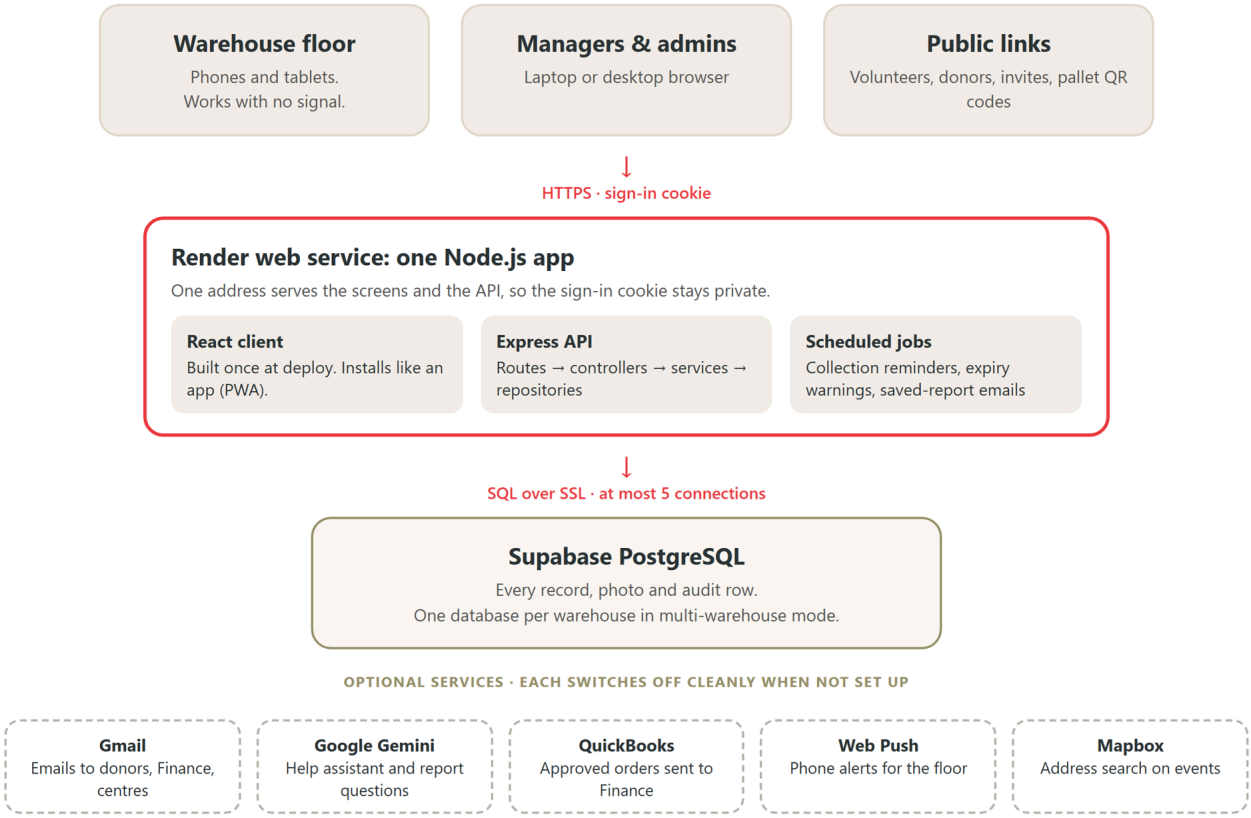
1. Get access to the GitHub repository, the Render service and the Supabase project (section 4).
2. Run it on your machine (section 8.1) and read `CODEBASE.md` in the repository.
3. Follow one feature from screen to database (section 7.4).
4. Check **Settings › Connections** in the live app: it shows what is switched on and working.
5. Read the open decisions in section 13, and the three decisions before going live in section 3.

02

SYSTEM OWNER

What it runs on

Everything runs as one web service in front of one database. The other services are optional: each one adds a feature, and the system keeps working without it.



- **One address for everything.** The sign-in cookie is set to stay on the site that issued it. Splitting the screens and the API across two addresses would make every request after sign-in fail, so they are deliberately served together.
- **Optional services switch off cleanly.** With no Gemini key the help assistant and report questions are hidden; with no push keys the bell still works; with no QuickBooks keys Finance is emailed instead.
- **Who sees what** is set once in `client/src/routes/routeTable.js` (the menu and page access) and enforced again on every server route with `requireRole(...)`.

Every service it uses

Service	What it does for Batches	Needed?	Without it
Render	Hosts the web service: builds the screens, runs the server, gives the HTTPS address, restarts it if it stops	Required	No system. Any Node.js 22 host works the same way (section 13)

Service	What it does for Batches	Needed?	Without it
Supabase	Managed PostgreSQL: every record, photo and the audit log	Required	No system. Any PostgreSQL 15+ works with a connection string
GitHub	Holds the code; runs the automated checks; Render deploys from it	Required	No way to deploy changes
Gmail (Google Cloud OAuth app)	Sends every email: invites, password resets, orders to Finance, donor thank-yous and Section 18A forms, collection reminders, scheduled reports	Strongly advised	Emails are recorded in Message history but not sent; nobody new can be invited by email
Google Gemini	The help assistant and the plain-words report questions	Optional	Both hidden; report questions fall back to keyword matching
Web Push (VAPID keys)	Phone alerts for new pallets	Optional	The in-app bell still works
QuickBooks Online (Intuit app)	Sends each approved purchase order to QuickBooks	Optional	Finance is emailed each order instead
Mapbox	Address search when placing centres on the Impact map	Optional	Addresses are typed by hand
Open Food Facts	Fills in a product from its barcode. Only the barcode is sent	Optional	Products are entered by hand

What the warehouse needs

FLOOR DEVICES

Phones or tablets

Any recent Android or iPhone with an up-to-date Chrome or Safari and a camera (for QR codes and photos). Install from the browser with *Add to Home screen*. A shared tablet per job station works well.

OFFICE

Laptop or desktop

Any current browser (Chrome, Edge, Firefox, Safari) for managers and admins. A printer for picking slips, pallet labels and notes, if paper is still wanted.

CONNECTIVITY

Wi-Fi or mobile data

The floor tolerates dead spots: work queues on the phone. Claiming a pallet needs a signal, so good Wi-Fi at the packing area and the gate helps most.

No barcode scanners are needed. The system reads pallet QR codes with the phone's camera, and product barcodes can be added later without changing how the floor works.

03

SYSTEM OWNER

What it costs

Batches was built and tested on free plans. They are fine for training and trials, but three of their limits matter for a warehouse that depends on the system every day.

The free set-up today, and its limits

Limit	What the warehouse would notice
Render free plan sleeps after 15 idle minutes	The first person in the morning waits about a minute for the page. **Scheduled jobs stop while it sleeps**: collection reminders, expiry warnings and saved-report emails are only sent while the service is awake.
Supabase free plan has no backups	If data is deleted or damaged, there is nothing to restore from except manual backups (section 9.3).
Supabase pauses a free project after a week with no activity	Over a long closure (for example December) the database can pause and must be restored from the dashboard before the system works.
Supabase free database is 500 MB	Photos are stored in the database. A few years of records plus photos will approach this.

Recommended live set-up

Item	Plan	About, per month
Render web service	Starter instance (always on, 512 MB). Standard (2 GB) only if it becomes slow	US$7 (Standard US$25)
Render workspace	Hobby is enough for one service; Pro if several people need dashboard access	Free (Pro US$25)
Supabase	Pro: 8 GB disk, daily backups kept 7 days, never paused	US$25
Point-in-time recovery	Optional Supabase add-on; restore to any minute	US$100 per 7 days kept
Gmail, Gemini, Mapbox, push	Free tiers cover this volume. Google Workspace for Nonprofits can provide the sending account	Free
Domain name	Optional, for example `batches.ladlesoflove.org.za`; a subdomain of the existing site costs nothing extra	Existing domain
Total	Render Starter + Supabase Pro	**≈ US$32 (about R600)**

Prices were checked in October 2026 from the providers' published pricing and change from time to time; confirm them before budgeting. Render's Hobby workspace includes 5 GB of outbound bandwidth a month, then US$0.15 per GB, which this system is unlikely to exceed. Both providers have programmes for non-profits worth asking about.

THREE DECISIONS BEFORE GOING LIVE

1. **Move Render to an always-on plan**, so the first page is instant and reminders and expiry warnings run on time.
2. **Move Supabase to Pro** (or another host with daily backups), and practise one restore.
3. **Move every account into Ladles of Love's ownership** (section 4), so nothing depends on a student's login.

04

SYSTEM OWNER

Accounts and access

Batches depends on a handful of outside accounts. Each should belong to Ladles of Love, be signed into with an organisation email (for example it@ or ops@), have two-step verification on, and have at least two people who can get in.

Account	What it holds	Hand over by
GitHub	The code and its history, the automated checks	Transfer the repository to a Ladles of Love organisation (Settings › Transfer). Add the maintainer as a collaborator
Render	The running service and its secret settings	Invite the organisation account to the workspace, or create the service again from `render.yaml` in an organisation-owned workspace and copy the settings across
Supabase	The database and its backups	Transfer the project to an organisation-owned Supabase organisation, or invite the organisation account as Owner
Google Cloud	The OAuth app Batches uses to send Gmail	Create the OAuth client under the organisation's Google account; an admin then connects Gmail in Settings
Gmail sending account	The mailbox system emails come from	Use an organisation mailbox, never a personal one
Google AI Studio	The Gemini key	Create a key under the organisation account; paste it into Render
Intuit Developer	The QuickBooks app (if used)	Create under the organisation's QuickBooks login
Mapbox	The public address-search token	Create under the organisation account; restrict the token to the app's address
Domain / DNS	The web address (if a custom domain is used)	Whoever already manages ladlesoflove.org.za
Batches admin login	The first admin user inside the app	Each warehouse's admin has their own login; never share one

AFTER THE HANDOVER

- Remove the student team from GitHub, Render, Supabase and Google once the organisation has taken over.
- Change `JWT_SECRET` (everyone signs in again) and the database password, and update `DATABASE_URL` in Render.
- Deactivate any test or student user accounts inside Batches.
- Keep this list somewhere the organisation will find it, with who holds each login.

05

TECHNICAL

Setting it up and going live

The steps from nothing to a working warehouse. The same steps rebuild the system on a new account, or set up a separate copy for a trial.

5.1 Infrastructure

1. **Database.** Create a Supabase project in the region nearest Cape Town that the plan allows. Copy the *session pooler* connection string, and download the CA certificate (Project Settings › Database › SSL Configuration).
2. **Schema.** Copy the existing database in Supabase, or, for a new empty warehouse, run `npm run migrate -- init <code>` in multi-warehouse mode (section 11). `schema.sql` is a reference, not a script.
3. **Web service.** In Render, create a Blueprint from the repository; it reads `render.yaml`. Point it at the branch you release from.
4. **Secrets.** Paste every `sync: false` value into Render's Environment tab, and add the settings `render.yaml` does not list (table below).
5. **Address.** Optionally add a custom domain in Render and a CNAME at the DNS host. Then set `APP_BASE_URL` to the final address.
6. **Check.** Open `/api/health`, then run `npm run security-check -- https://your-address` from `server/`. It only reads, and is safe against the live site.

Settings to add in Render that render.yaml does not list

Setting	Value
`APP_BASE_URL`	The public address, no trailing slash. Every link in an email starts with it; without it, emails with links are not sent
`GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`	From the Google Cloud OAuth client (Web application)
`GMAIL_REDIRECT_URI`	`https://your-address/api/gmail/callback`, registered exactly the same on the OAuth client
`GMAIL_TOKEN_ENCRYPTION_KEY`	Base64 of 32 random bytes. Keep it: changing it means connecting Gmail again
`VITE_MAPBOX_PUBLIC_TOKEN`	Public, address-restricted Mapbox token. Read at **build** time, so redeploy after setting it
`EMAIL_FROM`, `EMAIL_LOGO_URL`	Optional: the From address and the logo shown in emails

Setting	Value
`GATE_CODE_SECRET`	Optional: the key for the volunteers' daily gate code (otherwise derived from `JWT_SECRET`)

Leave `VITE_API_URL` unset in production: the screens then call the API on their own address, which is what the one-service design needs.

5.2 Inside the app

1. Sign in as the first admin (created with `npm run warehouse-admin -- create-admin`, or already present in a copied database).
2. **Settings › Email**: connect the organisation's Gmail account and set the Finance address. **Settings › Connections** should then show email, AI, push and QuickBooks as working or deliberately off.
3. **Master data**: products (with weight, cost, reorder level, decantable), suppliers and the products each supplies. `seedWesternCapeSupply.mjs` and `loadRealData.mjs` load the Cape Town data; both have `--dry-run`.
4. **Beneficiaries**: each centre with its cohort, child count and contact; **Centre locations** places them on the Impact map.
5. **Settings**: recipes (summer and winter), child bands, unit weights, expiry warnings, reminder times, Section 18A certificate details. **Operating calendar**: cohort days and closures.
6. **Warehouse layout**: draw the stores and staging areas, then count opening stock into them.
7. **Users**: invite managers and warehouse staff. Each receives an email to choose a username and password.
8. Run one full week in parallel with the old process before switching off the spreadsheets.

06

TECHNICAL

Configuration reference

Every setting the server reads is listed with its default in `server/env.example`; the client's are in `client/env.local.example`. This is the short version.

Setting	What it does	Without it
`DATABASE_URL`	Supabase session pooler connection string	Server will not start
`JWT_SECRET`	Signs sign-in sessions. Render generates it	Server will not start
`NODE_ENV=production`	Serves the built screens, secure cookies, strict origins	Development behaviour
`DB_SSL` , `DB_SSL_CA`	Encrypted, verified database connection	Encrypted but the certificate unchecked
`DB_POOL_MAX`	Connections per server (default 5; use 3 when developing)	Default
`TRUSTED_CLIENT_IP_HEADER`	`cf-connecting-ip` on Render, so rate limits see real visitors	Rate limits lump visitors together
`APP_BASE_URL`	The address every emailed link starts with	Emails with links are held back
`GEMINI_API_KEY` , `GEMINI_MODEL` , `GEMINI_FALLBACK_MODELS`	The AI help and report questions, and backup models when one is busy	Both features hidden
`GMAIL_*` , `EMAIL_ENABLED`	Sending email	Emails logged, not sent
`VAPID_PUBLIC_KEY` , `VAPID_PRIVATE_KEY` , `VAPID_SUBJECT`	Phone alerts. Generate once: `npx web-push generate-vapid-keys`	No phone alerts
`QUICKBOOKS_*`	Approved orders to QuickBooks; `QUICKBOOKS_ENVIRONMENT` is sandbox until set to production	Finance emailed instead
`WAREHOUSE_DB_URLS` , `WAREHOUSE_NAMES`	Multi-warehouse mode (section 11)	One warehouse on `DATABASE_URL`
`ECD_REMINDER_SCHEDULER` , `SAVED_REPORTS_SCHEDULER`	Set `false` to stop a job on one server	Both run
`OPEN_FOOD_FACTS_ENABLED`	Barcode look-up	On by default

Setting	What it does	Without it
`VITE_WAREHOUSE_NAME`, `VITE_WAREHOUSE_ADDRESS`	Letterhead on printed notes (build time)	Cape Town warehouse

Values an admin can change while the app runs (expiry warnings, reminder times, unit sizes, recipes) are not environment settings: they live in Settings, defined in `server/src/features/settings/settingsDefinitions.js`. Do not set `TZ`: dates are worked out in South African time in code.

07

TECHNICAL

The codebase

One repository, two halves, and a small set of rules that tell you where any piece of code belongs. `CODEBASE.md` in the repository goes through every folder.

7.1 Where things are

Folder	What is in it
`client/src/pages/`	One file per screen. Kept thin: a page lays out feature components
`client/src/features/<area>/`	Everything for one area (receiving, packing, reporting and so on), in one flat folder
`client/src/components/`	`ui/` : shared building blocks. `layout/` : menus and the shells for each role
`client/src/services/`	The only code that talks to the server. One file per area, plus the offline outbox
`client/src/routes/`	Every screen's address, who may open it and which menu it is in
`client/src/translations/`	Afrikaans and isiXhosa for the floor
`server/src/routes/`	Each address and the roles allowed to call it
`server/src/controllers/`	Read the request, call a service, send the answer
`server/src/services/`	The business rules. Refuses bad requests with a clear message
`server/src/repositories/`	All the SQL
`server/src/features/`	Calculations and catalogues the layers share: reports, units, recipes, settings
`server/src/integrations/`	Everything that talks to an outside system: email, PDFs, the volunteer system
`server/src/jobs/`	The three scheduled jobs
`server/database/`	`schema.sql` (every table) and numbered migrations
`server/scripts/`	Command-line tools: migrations, multi-warehouse admin, data loaders, security check

7.2 The rules the code follows

- **Four server layers**, each calling only the one below: route → controller → service → repository. SQL appears only in repositories. A service may open a transaction and hand its connection to the repositories it calls, so several writes succeed or fail together.

- **Same name in every layer.** To follow purchase orders, open `purchaseOrder.routes.js`, `.controller.js`, `.service.js` and `.repository.js`.
- **One list, one place.** Roles, purchase-order statuses, donation categories and storage areas each live once in `server/src/constants/`. Report metrics live once in `features/reporting/reportCatalog.js`.
- **Screens never call the server directly**; they go through `client/src/services/`.
- **Every change is recorded.** Writes add a row to `audit_log` in the same transaction, never with a password or token in it.
- **Record rather than block.** Business rules flag warehouse realities (a short delivery, a late collection) for a manager instead of refusing them; hard refusals are kept for genuine blockers.

`CODEBASE.md` lists the three deliberate exceptions to the layering rule and where new work belongs.

7.3 Cross-cutting pieces

Concern	Where it lives
Sign-in and roles	`server/src/middleware/auth.middleware.js`, `client/src/context/AuthContext.jsx`
Working with no signal	`client/src/services/outbox.js` keeps work on the phone; `server/src/middleware/idempotency.middleware.js` makes sure a resent request is only done once
Stock	`stock.repository.js` `adjustStock`: the balance and the ledger line change together. Stock comes off at dispatch, not packing
Notifications	`features/communications/notices.js`, written inside the same transaction as the change they announce
Emails	`communications.service.js` through the connected Gmail account; every send is logged in Message history
Settings an admin can change	`server/src/features/settings/settingsDefinitions.js` (defaults and limits)
Several warehouses	`server/src/config/dbRouter.js` and `warehouseContext.js` pick the database for each request

7.4 Follow one feature: receiving a delivery

Step	File
The screen	`client/src/features/receiving/ReceivingFlow.jsx` (and `ReceivingParts.jsx`)
The call	`client/src/services/receivingAPI.js` `recordDelivery`
The address and role check	`server/src/routes/delivery.routes.js`
The request	`server/src/controllers/delivery.controller.js`

Step	File
The rules	`server/src/services/delivery.service.js`
The SQL	`server/src/repositories/delivery.repository.js` `createDelivery`
The stock ledger	`server/src/repositories/stock.repository.js` `adjustStock`
The tests	`server/__tests__/` and `client/src/tests/`

7.5 Tests

- `cd client && npm run lint && npm test`, and `cd server && npm test`. Both must pass before merging.
- `server/__tests__/fixtures/routeRoles.baseline.json` records who may call every address. Changing a route's roles fails a test until you update that file on purpose.
- Server tests use a mocked database. `npm run test:integration` runs some against a real one; never point it at the live database.
- GitHub Actions lints and builds the client and runs the server tests on every push. It does not run the client tests yet; run them yourself.
- Approving a purchase order through the API sends a real email if Gmail is connected: test against a database with email off (`EMAIL_ENABLED=false`).

08

TECHNICAL

Running it and releasing changes

You need Node.js 22 and a Postgres database to point at: a copy, never the live one.

8.1 On your machine

Step	Command
Server settings	`cd server`, copy `env.example` to `.env`, fill in at least `DATABASE_URL` and `JWT_SECRET`. Set `DB_POOL_MAX=3`
Start the server	`npm install` then `npm run dev` (http://localhost:5000)
Client settings	`cd client`, copy `env.local.example` to `.env.local`
Start the client	`npm install` then `npm run dev` (http://localhost:5173)

The code is developed on Windows and Linux. Keep file names unique ignoring capital letters (Windows treats `Item.jsx` and `item.jsx` as the same file), and let Git handle line endings.

8.2 Hosting on Render

- **What runs:** one Render web service built from `render.yaml` in the Frankfurt region (the nearest to Cape Town). The build compiles the client; Express then serves it and the API from one address.
- **Which branch:** `staging/(DEVELOPMENT-TESTING)` today. Point Render at `main` once that is the branch you release from.
- **Health check:** `/api/health`. Render restarts the service if it stops answering.
- **Secrets** are pasted into the Render dashboard, never committed: `DATABASE_URL`, `GEMINI_API_KEY`, the Gmail, QuickBooks and push keys. `JWT_SECRET` is generated by Render.
- **Free plan:** the service sleeps after 15 idle minutes and takes about a minute to wake. Move to a paid plan before relying on it every day (section 3).

8.3 Releasing a change

Branch A `feature/…` branch from the release branch › **Test** Lint, client and server tests pass locally › **Review** Pull request; CI must be green › **Migrate** Back up, then `migrate --up` if the database changes › **Merge** Render builds and switches over › **Check** `/api/health`, Settings › Connections, one real task

DEPLOYING A CHANGE

Merge to the deployed branch. Render builds the new version, then switches over to it. If the change has a database migration, run it **before** the deploy (section 9.2), so the new code never meets an old database. If a release goes wrong, Render's *Rollback* returns to the previous build in a click; a migration is undone by a new migration, not by editing the old one.

BEST TIME TO RELEASE

Not on a collection day (Tuesday or Thursday) and not in the morning rush. A Monday or Wednesday afternoon gives the warehouse time to report problems before the next dispatch.

09

TECHNICAL

The database

The database lives in Supabase and is the real thing. Everything the warehouse knows is in it, so it gets the most care.

9.1 What it holds

- **Supabase PostgreSQL.** `server/database/schema.sql` describes every table; read it, do not run it.
- **Photos** (flagged items, deliveries) are stored in the `photos` table, shrunk to about 1,280 pixels. There is room for a few thousand. To move them to file storage, change only `photo.repository.js`.
- **Connections:** the Supabase session pooler allows 15 in total. Each server keeps at most 5 (`DB_POOL_MAX`). A live server, a deploy starting beside it and two developers fill it: developers should use 3.
- **Personal data:** staff names and emails, centre contacts, donor names and emails, volunteer names. Feed the Soil keeps only a name and suburb. See section 12.

9.2 Changing the database

- Add the next numbered file to `server/database/migrations/` (the latest is `060`, so the next is `061_what_it_does.sql`), and make the same change in `schema.sql`.
- Never change one warehouse's database by hand. Every site must run the same schema.

Command (in `server/`)	What it does
`npm run migrate -- status`	What is applied and what is waiting, for every database
`npm run migrate -- up`	Applies waiting migrations to every database. Checks all are reachable first, runs each file in its own transaction, and stops at the first failure
`npm run migrate -- init <code>`	Builds a new, empty warehouse database

9.3 Backups

CHECK THIS FIRST

- Confirm the Supabase plan's backups and how far back you can restore. The free plan has no backups and no point-in-time recovery; Pro keeps daily backups for 7 days.
- Take a manual backup (`pg_dump` , or Supabase's download) before every migration and every bulk data load.
- Keep a monthly copy somewhere other than Supabase (for example the organisation's Google Drive), since 7 days is short.
- Practise a restore into a spare project once, so you know it works before you need it.

9.4 How big it will grow

Records are small: a year of deliveries, slips, dispatches and donations for one warehouse is tens of megabytes. Photos are the part that grows. Watch the database size in Supabase each month; when it passes about two-thirds of the plan's limit, move photos to Supabase Storage (section 13) or move up a plan.

10

TECHNICAL

Looking after it

A healthy Batches needs about an hour a month of attention. Most of it is checking, not fixing.

10.1 Routine tasks

How often	Task	Who
Every week	Look at Settings › Connections (email, AI, push, QuickBooks) and Message history for failed emails	Admin
Every month	Read Render's logs for errors. Check Supabase database size and connection use. Ask an admin to deactivate people who have left. Take an off-site backup	Maintainer
Every quarter	Update dependencies (`npm outdated` and `npm audit` in `client/` and `server/`), run all tests, deploy. Test a backup restore. Run `npm run security-check`	Maintainer
Every year	Check the Gemini model names in `render.yaml` are still offered, and that the Gmail and QuickBooks connections have not expired. Review who has access to every account (section 4)	Maintainer and system owner

10.2 Scheduled jobs

Job	When	Switch
Collection reminders to centres	Daily at the hour set in Settings	`ECD_REMINDER_SCHEDULER=false` turns it off on that server
Expiry warnings, idle Feed the Soil kits, QuickBooks retries	Checked hourly, acts once a day	Always on. Safe to run twice: it skips anything already sent
Saved-report emails	On each report's schedule	`SAVED_REPORTS_SCHEDULER=false`

> **THE JOBS LIVE INSIDE THE WEB SERVICE**
>
> There is no separate scheduler. The jobs run only while the service is awake, which on the free Render plan means only after someone has used the system. On an always-on plan they run on time.

10.3 Secrets and keys

Secret	If you change it
`JWT_SECRET`	Everyone is signed out and signs in again. Change it at once if it may have leaked. Today's gate code also changes unless `GATE_CODE_SECRET` is set
`GMAIL_TOKEN_ENCRYPTION_KEY`	The Gmail account must be connected again in Settings
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY`	Every phone must turn alerts on again. Do not rotate casually
`DATABASE_URL`	Update it in Render; the service restarts
`GEMINI_API_KEY`	No side effects. Free-tier limits are per model; backup models are listed in `GEMINI_FALLBACK_MODELS`

10.4 When something goes wrong

Symptom	Likely cause and fix
First page takes about a minute	Free Render plan waking up. Upgrade the plan
Nothing loads at all; Supabase shows the project paused	Free project paused after a week idle. Restore it in the Supabase dashboard; move to Pro
"max clients reached" at start-up	Database connections full. Lower `DB_POOL_MAX` on developer machines, or stop a stray server
Help assistant or report questions missing	`GEMINI_API_KEY` not set, or the model retired. Check Settings › Connections
Emails not arriving	Gmail disconnected, `APP_BASE_URL` missing, or `EMAIL_ENABLED=false`. Check Message history and reconnect Gmail
Centres not reminded / no expiry warnings	The service was asleep at the hour (free plan), or the scheduler switch is off
Signed in, then every request fails	Screens and API served from different addresses. Keep them on one (section 2)
A floor worker's saves are stuck	They are queued on the phone. They send when the signal returns; the bar at the top lists them
Address search on the Impact map does nothing	`VITE_MAPBOX_PUBLIC_TOKEN` missing at build time, or restricted to another address. Set it and redeploy
Build fails: cannot find `@vitejs/plugin-react`	The client install must include dev dependencies (`--include=dev` , already in `render.yaml`)

Render's *Logs* tab shows the server's errors; Supabase's *Logs* and *Reports* show database errors and load. The admin's Activity log and Message history show what users did and what was sent.

11

TECHNICAL

Bringing in another warehouse

Ladles of Love has warehouses in Cape Town, Gauteng and the Northern Cape. They run the same processes but share no data: pricing, goods and suppliers are regional. A second site can join in one of two ways. Both keep each site's data in its own database.

	A. Its own copy	B. One app, several warehouses
How	A second Render service and Supabase project	The same service; add the site to `WAREHOUSE_DB_URLS`
Best for	A separate organisation, or a trial	Ladles of Love sites that share staff Recommended
Staff at two sites	Two accounts	One sign-in, with a warehouse switcher
Updates	Deploy and migrate each copy	One deploy; `migrate -- up` updates every site
Cost	Hosting and database per site	One service; one database per site

11.1 Adding a site to one app (option B)

1. Create a new Supabase project for the site and copy its connection string.
2. Add it to `WAREHOUSE_DB_URLS` (a JSON list keyed by a short code, for example `"gauteng"`) and its display name to `WAREHOUSE_NAMES`. Do this in your local `.env` for the next two steps too.
3. `npm run migrate -- init gauteng` builds the empty database with every table.
4. `npm run warehouse-admin -- create-admin gauteng --username … --first … --last …` makes its first admin and prints a one-time password to hand over.
5. Put the same `WAREHOUSE_DB_URLS` in Render and deploy. Staff at more than one site: `warehouse-admin -- grant`.
6. Guests at that site sign in with `?w=gauteng` on the sign-in link, or choose the site on the sign-in page.

> **CONNECTIONS WITH SEVERAL SITES**
>
> Each site's database has its own connection pool. With three sites the server holds up to three pools, so check each Supabase project's connection limit, and keep `DB_POOL_MAX` modest.

11.2 Setting the site up

- **Admin, in the app:** products, suppliers, beneficiaries and their cohorts, recipes, users; Settings for cohort days, unit weights, expiry warnings, reminder times, Section 18A certificate details; connect a Gmail account. Follow section 5.2.
- **Per deployment (option A only):** `VITE_WAREHOUSE_NAME` and `VITE_WAREHOUSE_ADDRESS` for the printed-note letterhead, `EMAIL_LOGO_URL`, and `APP_BASE_URL`.
- **Fixed in code today:** receiving offers two storage places (dry store and cold room), and the "near expiry" lists use 14 and 30 days. A site that needs different values needs a small code change (section 13).

12

EVERYONE

Security and data protection

Batches holds personal information about staff, centres, donors and volunteers. The system protects it by design; the organisation's part is to look after the accounts and keys around it.

Protection	How
Passwords	Hashed with bcrypt. The hash is never sent to a screen or written to a log or audit row
Sign-in session	A signed token in an httpOnly, secure, same-site cookie that page scripts cannot read. Lasts 8 hours (12 for a volunteer)
Who may do what	Every server route checks the role. A test holds every route to a recorded list
Accounts stay current	Each visit re-reads the account, so deactivating someone or changing their role takes effect at once
Too many attempts	Sign-in, password reset, invites, account changes and public links are rate-limited (for example 10 sign-ins per 15 minutes)
Changing your own details	Email and password changes need the current password
Links in emails	Invites, password resets and Section 18A forms use random tokens stored only as a hash. Resets work once, for one hour
Volunteers	A daily six-digit gate code, or the code on a pallet, is needed to open a guest session
Browser protections	Helmet security headers and a content security policy. CORS limited to the app's own address
Connected accounts	Gmail and QuickBooks tokens are encrypted in the database (AES-256-GCM)
Records	`audit_log` records who changed what. Nothing is hard-deleted that a record points at: people and products are deactivated or archived
Data between sites	Each warehouse has its own database. Requests are checked against the warehouses a person has access to
Logs	Personal details are removed from server logs

KEEP IT SECURE

- Keep secrets in Render and in local `.env` files only. `.env` is ignored by Git; keep it that way.
- Give the Supabase and Render dashboards to as few people as possible, each with their own login and two-step verification.
- Run `npm audit` with each dependency update.
- `/api/health` and the public links are the only things reachable without signing in.

POPIA

- Collect only what the work needs; the screens already do (Feed the Soil keeps no address or phone).
- Name an information officer at Ladles of Love responsible for this data.
- Use test copies of the database with care: they hold real people's details.
- If a key or database password may have leaked, rotate it (section 10.3) and review the audit log.

13

TECHNICAL

Growing it

Batches is built to grow without a rewrite. Most limits are lifted by a plan change or a change in one file.

13.1 Today's limits and what to do

Limit	Grows to	What to do
Free Render plan sleeps	Always on	Move to a paid Render plan. No code change
15 database connections	More users and servers	Use Supabase's transaction pooler or a larger plan, then raise `DB_POOL_MAX`
One server	Two or more	Works as is: sessions live in the cookie, and repeated requests are caught in the database. Turn the reminder and saved-report jobs off on all but one server
Photos in the database	Tens of thousands	Move them to Supabase Storage or S3; only `photo.repository.js` changes
Report speed	Years of data	Reports are cached briefly. Add indexes for slow reports, or a read replica for reporting
More sites	Many warehouses	Multi-warehouse mode (section 11). Each site's database grows on its own
Another host	Any Node.js 22 host	Nothing is Render-specific beyond `render.yaml`: build the client, start the server, set the same environment, keep one address
Barcode scanners	Scanning at receiving	Products already hold a barcode; a scanner acts as a keyboard, so the receiving search can take a scan with little change

13.2 Open decisions and loose ends

- Receiving offers 2 of the 5 storage areas, and "near expiry" uses fixed 14 and 30 days rather than the Settings values. Decide whether these should follow Settings.
- The decanting bag sizes are a fixed list, kept once on each side (`BagSizes.jsx`, `decanting.service.js`).
- The Afrikaans and isiXhosa translations are drafts. Ask a fluent speaker to check `client/src/translations/` before relying on them; only part of the floor is translated so far.
- CI does not run the client tests. Adding the step needs GitHub permission to edit workflows.
- The deployed branch is `staging/(DEVELOPMENT-TESTING)`. Agree a release branch (`main`) and point Render and CI at it.

13.3 A support model for a team without IT staff

INSIDE LADLES OF LOVE

A system owner and a super-user

The system owner holds the accounts and the budget. A super-user per warehouse (usually the admin) trains new staff, answers first questions from the in-app Help, and keeps master data tidy.

OUTSIDE

A maintainer on a small retainer

A freelance developer, IT partner or a future UCT team does the monthly checks, quarterly updates and small changes. Any developer who knows React, Node.js and PostgreSQL can pick it up from this document and `CODEBASE.md`.

13.4 Where to read more

Document	For
`README.md`	Setting up, scripts, things to know
`CODEBASE.md`	Every folder, where new work goes, the layering rule
`server/env.example`, `client/env.local.example`	Every setting
`render.yaml`	Hosting, with notes on each setting
`docs/Batches - How to use the system`	Training staff, managers and admins

Built for Ladles of Love by UCT INF3003W Team 22, 2026. Hosting prices from Render and Supabase's published pricing, October 2026. When the code and this document disagree, the code and `CODEBASE.md` are the source of truth.